

TRIPLE DES

para Python

► **Areli Alejandra Guevara Badillo**

► Miércoles, 23 de octubre de 2025

PyCripodome

PyCripodome es una biblioteca de Python diseñada para ofrecer herramientas de criptografía de alto nivel y bajo nivel. Permite implementar algoritmos de cifrado, generación de claves, firmas digitales y funciones hash de manera sencilla.

Es una bifurcación moderna de PyCrypto, con mejoras en rendimiento, seguridad y compatibilidad con versiones recientes de Python. Se utiliza ampliamente en proyectos que requieren proteger información o realizar operaciones criptográficas seguras.

Especificaciones básicas

- **Nombre del paquete:** pycryptodome
- **Lenguaje:** Python (compatible desde la versión 3.6 en adelante)
- **Funciones principales:**
 - Cifrado simétrico (AES, DES, 3DES, ChaCha20, etc.)
 - Cifrado asimétrico (RSA, DSA, ECC)
 - Hashes criptográficos (SHA, MD5, HMAC, etc.)
 - Generación de números aleatorios seguros

La instalación se realiza directamente desde **pip**, el gestor de paquetes de Python.

En una terminal o consola se ejecuta el siguiente comando:

```
pip install pycryptodome
```

Cifrado 3DES en PyCripodome

Especificaciones de 3DES en PyCripodome

Modo interno que utiliza

PyCripodome implementa el algoritmo **3DES en modo EDE (Encrypt–Decrypt–Encrypt)**.

Esto significa que los datos son cifrados, luego descifrados con la segunda clave, y finalmente cifrados otra vez con la tercera clave.

El modo EEE (Encrypt–Encrypt–Encrypt) no se utiliza en esta biblioteca.

Tamaño de la llave

PyCripodome acepta **llaves de 16 o 24 bytes**, equivalentes a:

- **128 bits (16 bytes):** Usa dos llaves distintas (K1, K2, K1).
- **192 bits (24 bytes):** Usa tres llaves distintas (K1, K2, K3).

Tipo de clave

- **2-key 3DES:** Clave de 16 bytes (usa dos claves diferentes).
- **3-key 3DES:** Clave de 24 bytes (usa tres claves diferentes).

El módulo valida automáticamente que la clave tenga la **paridad correcta** usando `DES3.adjust_key_parity()`.

¿Cómo acceder al cifrado 3DES?

Para utilizar **3DES (Triple Data Encryption Standard)** dentro de PyCryptodome, se emplea el módulo `Crypto.Cipher.DES3...`

Importar la clase DES3

```
from Crypto.Cipher import DES3
```

Generar o definir una clave válida

```
from Crypto.Random import get_random_bytes
key = DES3.adjust_key_parity(get_random_bytes(24))
```

Cifrar y descifrar

Función para cifrar con Triple DES en modo CBC

Para realizar operaciones de cifrado con el algoritmo **3DES en modo CBC (Cipher Block Chaining)**, la biblioteca PyCryptodome utiliza la clase `DES3`, contenida en el módulo `Crypto.Cipher`.

La creación del objeto de cifrado se realiza mediante la función `DES3.new`:

```
from Crypto.Cipher import DES3
from Crypto.Random import get_random_bytes

key = DES3.adjust_key_parity(get_random_bytes(24))
iv = get_random_bytes(8)
cipher = DES3.new(key, DES3.MODE_CBC, iv)
```

Parámetros requeridos

Parámetro	Descripción
key	Clave de 16 o 24 bytes. Se ajusta la paridad mediante <code>DES3.adjust_key_parity()</code> .
mode	Modo de operación, en este caso <code>DES3.MODE_CBC</code> .
iv	Vector de inicialización de 8 bytes obligatorio en el modo CBC.

Mecanismo de relleno (Padding)

El algoritmo 3DES trabaja con **bloques de 8 bytes**, por lo que los mensajes deben ser múltiplos de esta longitud.

PyCryptodome no aplica un esquema de relleno de forma automática, por lo que debe emplearse el módulo `Crypto.Util.Padding`, que utiliza el estándar **PKCS#7** para ajustar el tamaño de los datos antes del cifrado.

```
from Crypto.Util.Padding import pad, unpad

data = b"Mensaje secreto"
```

```

padded_data = pad(data, 8)
ciphertext = cipher.encrypt(padded_data)

```

El mecanismo **PKCS#7** añade bytes de relleno al final del texto para completar el bloque. Durante el descifrado, dichos bytes se eliminan utilizando la función `unpad()`.

Función para descifrar con Triple DES en modo CBC

El proceso de descifrado se realiza con la misma clase `DES3`, utilizando idénticos parámetros de configuración: la clave, el modo de operación y el vector de inicialización original.

El método utilizado para revertir el cifrado es `decrypt()`, seguido del proceso de eliminación del relleno mediante `unpad()`.

```

decipher = DES3.new(key, DES3.MODE_CBC, iv)
decrypted_data = decipher.decrypt(ciphertext)
original = unpad(decrypted_data, 8)
print(original.decode())

```

Parámetros requeridos

Parámetro	Descripción
key	Misma clave utilizada en el cifrado.
mode	Modo de operación CBC (<code>DES3.MODE_CBC</code>).
iv	Mismo vector de inicialización empleado en el cifrado.
ciphertext	Datos cifrados a descifrar.

Ejemplo de uso

```

1 from Crypto.Cipher import DES3
2 from Crypto.Random import get_random_bytes
3 from Crypto.Util.Padding import pad, unpad
4
5 key = DES3.adjust_key_parity(get_random_bytes(24))
6 iv = get_random_bytes(8)
7
8 cipher = DES3.new(key, DES3.MODE_CBC, iv)
9
10 data = b"Ejemplo de cifrado con Triple DES"
11 ciphertext = cipher.encrypt(pad(data, 8))
12 print("Texto cifrado:", ciphertext.hex())
13
14 decipher = DES3.new(key, DES3.MODE_CBC, iv)
15 plaintext = unpad(decipher.decrypt(ciphertext), 8)
16 print("Texto original:", plaintext.decode())

```

```

[Running] python -u "c:\Users\Areli Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escritura\Criptografia\Tarea 3\pycripto.py"
Texto cifrado: cc17c95f23b3e6acbecc5248500821b1bc044ee1538b6e017f9aed3c39d6883b3cdb188dff0dc89
Texto original: Ejemplo de cifrado con Triple DES

```